UNIT 10 60 min

Libraries and Modules

Import the standard library, read its documentation, and split your own code into modules.

By the end of this unit you will be able to

- Import a module and call its functions with dot notation
- Use `from ... import` and `import ... as` and explain when each is appropriate
- Find a function's documentation with `help()` and `dir()`
- Explain what the standard library includes and what `pip` adds
- Create a module file and import it from another file

Modules and libraries

A **module** is a file of Python code that can be imported. A **library** is a collection of modules. Python ships with a large **standard library**, so many common tasks need no installation. Anything else you install with `pip`, which is covered in the NumPy and pandas units.

Importing

imports.py

```
import math
import statistics

print(math.sqrt(16))
print(math.pi)
print(math.floor(3.7))
print(statistics.mean([2, 4, 6]))
```

After `import math`, everything in the module is reached with a dot: `math.sqrt`. The name `math` comes first so you always know where a function lives.

import_styles.py

```
from math import ceil, factorial
print(ceil(2.1))
print(factorial(5))

import math as m
print(m.hypot(3, 4))
```

> **Tip**
> Use `import math` when many names come from one module. Use `from math import ceil` when you need only one or two names. An alias such as `as m` is common for long names; pandas and NumPy use `pd` and `np` by convention.

Finding out what a module offers

discover.py

```
import math

print(len(dir(math)))          # how many names the module has
print(math.sqrt.__doc__)       # the documentation string
```

In an interactive environment, `help(math.sqrt)` prints the same documentation. Reading the documentation is a skill you will use every day. Get in the habit of checking the docs before guessing.

Some useful standard-library tools

Module	What it gives you	Example
`math`	Square root, rounding, trig, constants	`math.sqrt(9)` → 3.0
`statistics`	Mean, median, standard deviation	`statistics.median([1, 3, 9])` → 3
`random`	Random choices and numbers	`random.choice(["a", "b"])`

Module	What it gives you	Example
`datetime`	Dates and times	`datetime.date.today()`
`csv` / `json`	Read and write common data files	`json.loads(text)`

> **Note**
> Random results differ on every run. To get the same "random" numbers each time while you test, call `random.seed(42)` first. Seeding is standard practice in data science and machine learning.

Writing your own module

Any `.py` file is a module. Put a helpful function in its own file and import it from your main program. Two files are needed, so this example is shown as code to save on your computer rather than in the browser sandbox.

greetings.py (module file)

```
def hello(name):
    """Return a friendly greeting."""
    return f"Hello, {name}!"

if __name__ == "__main__":
    # Runs only when you run this file directly, not when it is imported.
    print(hello("test"))
```

main.py (in the same folder)

```
from greetings import hello

print(hello("Ada"))
```

The `if __name__ == "__main__":` line lets a file work both as a script and as a module. Python sets `__name__` to `"__main__"` only when the file is run directly.

Watch out

Do not name your file the same as a standard-library module (for example `random.py` or `math.py`). Python will import your file instead of the library and the program will fail in confusing ways.

Predict the output THINK FIRST

Remember what `round()` does with exactly .5, and that `math` functions return floats.

```
import math
print(math.floor(-2.5))
```

Answer: `-3`

```
import math
print(math.ceil(2.1))
```

Answer: `3`

```
from math import pow
print(pow(2, 3))
```

Answer: `8.0`

```
import math
print(math.factorial(5))
```

Answer: `120`

```
print(round(2.5))
```

Answer: `2`
Python rounds .5 to the nearest even number ("banker's rounding").

```
print(round(3.5))
```

Answer: `4`

```
import math
print(math.hypot(3, 4))
```

Answer: `5.0`

```
print(abs(-7))
```

Answer: `7`

Practice

▼ **Exercise 1 — Hypotenuse two ways**

For a right triangle with legs 5 and 12, compute the hypotenuse once with `math.sqrt` and once with `math.hypot`. Print both results.

▼ **Show solution**

Solution

```
import math

a = 5
b = 12
print(math.sqrt(a ** 2 + b ** 2))
print(math.hypot(a, b))
```

▼ **Exercise 2 — Build a module**

Create `shapes.py` with a function `circle_area(radius)` that returns `math.pi * radius ** 2`. In a second file, import it and print the area of a circle with radius 2. Write both files on your computer.

▼ **Hint**

The module file needs `import math` at the top.

▼ **Show solution**

Solution

```
# shapes.py
import math

def circle_area(radius):
    return math.pi * radius ** 2

# main.py
from shapes import circle_area
print(circle_area(2))
```

Discussion

1. Why do you think Python includes a large standard library instead of making everyone install packages? What are the trade-offs?
2. What is the risk of a file named `random.py` in your project folder? How would you find the cause of the error?
3. When you see `import numpy as np` in someone else's code, what do you expect the rest of the code to look like?

▼ **Facilitator notes**

Show students how to find a module's documentation online (docs.python.org) and how to check whether a name came from the standard library or from a file in their folder.

Key terms

Module

A Python file (`.py`) whose functions and variables can be imported into another program.

Library

A collection of modules that provides a set of related tools, such as the standard library, NumPy, or pandas.

Standard library

The modules that come with every Python installation, such as `math`, `random`, and `json`.

import

A statement that loads a module so its names can be used.

Alias

A shorter name given to a module on import, such as `import numpy as np`.

pip

The package installer for Python. It downloads third-party libraries from the Python Package Index.

Seed

A starting value for a random number generator. The same seed produces the same sequence every run.

__name__

A built-in variable. It equals `"__main__"` when a file is run directly, and the module name when imported.

See all terms in the glossary →